

PRIVANTRIX

AVANTA

Continuous Software Assurance

A private trust runtime concept for
evidence-driven software change.

CURRENTLY IN RESEARCH & DEVELOPMENT

Pranav Kumar Mishra

Project Lead

Aryan Kashyap

Co-Lead

Gagan Chaddah

Mentor / APS JRC

TRUST
EVERY
CHANGE.

AI adoption is high. Trust still needs evidence.

SOFTWARE CHANGE



REVIEW / TRUST?

84%

use or plan to use AI
in development

46%

distrust the accuracy
of AI tool output

66%

cite almost-right
AI answers as a frustration

WHO FEELS THE PRESSURE

Developers reviewing AI-generated PRs, startup CTOs and engineering leads with limited AppSec capacity.

Opportunity: one inspectable trust decision for each change

Source: Stack Overflow Developer Survey 2025, AI section. Self-reported responses to different questions.

Useful tools. A fragmented trust decision.

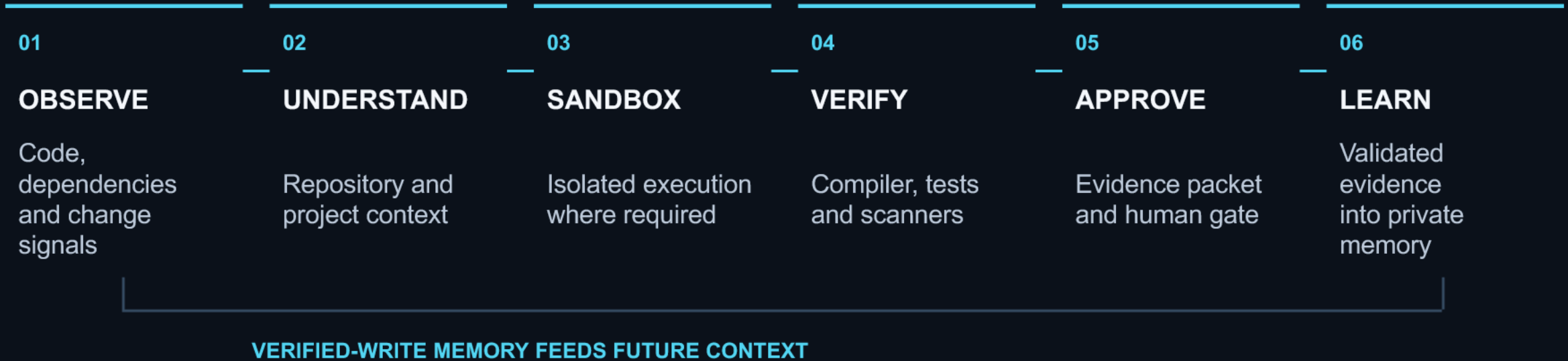


Proposed unmet need: a private assurance layer before release

Sources: GitHub Copilot, Code Scanning and Actions documentation; OpenTelemetry overview. Links in notes.

Evidence before trust

PROPOSED AVANTA TRUST LOOP



Pull request → context → evaluation → proposed action → verification → evidence → human decision

Reason probabilistically. Verify deterministically.

PROPOSED AVANTA ARCHITECTURE / CURRENTLY UNDER DEVELOPMENT

SOURCE

**Code
change
Pull request**

CONTEXT LAYER

Repository structure / dependencies
Program graph / security findings
Historical project memory

Structured context packet

RUDRA REASONING

Interpret context. Propose a patch.

PROPOSED CHANGE

A candidate action crosses the gate boundary

DETERMINISTIC VERIFICATION

Compile / unit + integration tests / SAST
Dependency checks / security behaviour
Sandbox execution where required

EVIDENCE PACKET — HUMAN GATE

Approve or reject against defined policy

Models propose. Evidence controls the trust decision.

Architecture first. Integrated MVP next.

CURRENT STATUS: RESEARCH & DEVELOPMENT

COMPLETED	IN PROGRESS	IN PROGRESS	NEXT	FUTURE
Research	Architecture	Fine-tuning	Integrated MVP	Controlled pilot
Concept defined	Context + gate design	Model / workflow trials	End-to-end loop	External validation

PLANNED MVP VALIDATION FLOW

Safe public / synthetic repo → controlled vulnerability → context → proposed change

Verification gates → evidence report → human approve / reject

Target: accept a change only after the required gates pass

Frequent releases. Limited AppSec capacity.

PROPOSED BEACHHEAD SEGMENT

10–200

engineers per team

Frequent releases
AI-assisted development
Limited dedicated AppSec staff

PRIMARY USERS

Startup CTO / engineering lead
Senior developer / security-minded founder
Small AppSec team

WORKFLOWS

AI-generated PR / dependency upgrade
Security patch / release candidate
Onboarding and engineering context

One assurance layer, with private context across reviews

Success needs a baseline and an evidence trail

PLANNED VALIDATION METRICS / NO RESULTS CLAIMED

TIME / EFFICIENCY

Less review effort

Triage minutes: finding to decision
Review hours per repository
Tool switches and manual handoffs

SECURITY / VERIFICATION

More complete evidence

Required gates satisfied (%)
Unsafe dependency exposure time
Failures, retries and false approvals

CONTEXT / AUDIT

Traceable decisions

Evidence-packet completeness
Validated context preserved
Decision and memory provenance

BASELINE

Measure the current workflow

PILOT

Repeat on comparable changes

TARGET

Set after baseline collection

Desired outcome: faster decisions with stronger evidence

A narrow workflow must earn its expansion

PROPOSED COMMERCIAL MODEL / REQUIRES CUSTOMER VALIDATION

STARTUP / PILOT

Per repository or app

Scanners + evidence + approval

TEAM

Per repo / workload / developer

CI + private memory + policy

ENTERPRISE / FUTURE

Private deployment + support

VPC / on-prem + audit controls

PILOT VALUE FRAMEWORK

Review hours saved × cost per hour
+ avoided remediation / delay cost
= estimated customer value

Measure compute cost per analysis, cost per verified change, retries and infrastructure cost.

STRATEGIC PRIORITY MATRIX

LOWER READINESS	INITIAL PRIORITY
Autonomous verified repair Multi-repo memory Policy intelligence	AI PR assurance Security evidence aggregation CI / release verification
Lower priority: general coding assistant, observability replacement, project management	

CURRENT Research + architecture MVP Single repo INTEGRATION CI + sandbox
TEAM RUNTIME Private memory + policy ENTERPRISE FUTURE Deployment + audit governance

Measurement framework, not financial performance. Proposed sequence then adds team memory and enterprise governance.

The assurance loop comes before the platform

BUILD NOW

Scanners / CI hooks / repository APIs
Containers / compilers / tests
Structured evidence reports

R&D REQUIRED

Context quality / memory safety
Model choice / fine-tuning / graphs
Secure access / gate orchestration

SCALE LATER

Single repository → team workspace
Multi-repository memory
Private enterprise runtime

RISKS / PROPOSED MITIGATIONS

Risk	Potential impact	Proposed mitigation
False positives	Wasted review time	Correlate signals + human triage
Incorrect model patch	Unsafe or broken change	Compiler + tests + security gates
Memory poisoning	Incorrect future context	Verified-write policy + provenance
Sensitive code exposure	Loss of confidentiality	Private/local design + scoped access
Over-automation	Unsafe autonomous decisions	Human approval for high-risk actions
Compute cost	Poor unit economics	Selective models + bounded workflows

Automate only what can remain verifiable

Proposed differentiation: private context + verification

DIRECTIONAL POSITIONING / AVANTA COLUMN SHOWS DESIGN INTENT

Capability	Coding agents	Security tools	CI/CD	Observability	AVANTA
Generate / suggest fixes	HIGH	MEDIUM	LOW	LOW	MEDIUM
Private repository context	HIGH	HIGH	HIGH	MEDIUM	HIGH
Persistent engineering memory	MEDIUM	MEDIUM	LOW	MEDIUM	HIGH
Verification workflow	HIGH	HIGH	HIGH	MEDIUM	HIGH
Human approval	HIGH	HIGH	HIGH	MEDIUM	HIGH
Cross-layer evidence	MEDIUM	MEDIUM	HIGH	HIGH	HIGH
Pre-release assurance	HIGH	HIGH	HIGH	MEDIUM	HIGH
Auditability	HIGH	HIGH	HIGH	HIGH	HIGH

PRIVATE CONTEXT + EVIDENCE + VERIFICATION + HUMAN GATE

POTENTIAL FUTURE DEFENSIBILITY

Orchestration depth, verified memory rules and accumulated engineering context

Every AI-generated and human change must present evidence

Team assessment of solution positioning. Capabilities vary by product. Deeper validation planned. Sources in notes.

Safer software should emerge from every change

Pranav Kumar Mishra

PROJECT LEAD

Architecture / product vision
Research direction

Aryan Kashyap

CO-LEAD

Engineering support / prototype
Testing workflow

Gagan Chaddah

MENTOR

Guidance / validation
Presentation review

STUDENT-LED / RESEARCH-DRIVEN / APS JRC

We are researching and building the foundations of software that stays secure, understandable, maintainable and verifiable as code, teams and threats evolve.

TRUST EVERY CHANGE.

Seeking technical mentors, validation feedback and safe repositories. Design partners after MVP validation.